DESENVOLVIMENTO DE API BACK-END · ENGENHARIA DE SOFTWARE · 6º PERÍODO

Semana 01
APIs, ecossistema Java e o método com IA

Apostila de estudo: teoria, exemplos, construção com IA e exercícios

Esta apostila acompanha a primeira aula. Ela apresenta os conceitos que sustentam todo o semestre: o que é uma API, como chegamos ao estilo REST, quais peças do ecossistema Java vamos usar e como trabalhar com um assistente de IA sem terceirizar o raciocínio. Leia antes ou depois da aula; os exercícios do fim ajudam a conferir o que ficou.

Conteúdo

1. Objetivos da semana
2. O que é uma API
3. Por que as APIs importam
4. Como chegamos aqui: de RPC a REST
5. O estilo REST
6. Quem pode usar a API: visibilidade
7. O ecossistema Java
8. Spring e Spring Boot
9. O caminho de uma requisição
10. Nomeando recursos
11. O projeto do semestre
12. Construindo com IA
13. Glossário
14. Exercícios
15. Para estudar

1. Objetivos da semana

↑ conteúdo

Ao final desta semana você deve ser capaz de:

1. explicar o que é uma API e diferenciar a interface (o contrato) da implementação;
2. situar SOAP, REST, GraphQL e gRPC na história das integrações e dizer quando cada um faz sentido;
3. descrever as restrições do estilo REST e reconhecer uma URI bem formada;
4. identificar o papel da JVM, do Maven, do Spring e do Spring Boot numa API Java;
5. usar os quatro movimentos do método com IA (Especificar, Gerar, Revisar, Provar) num pedido simples.

2. O que é uma API

↑ conteúdo

API (*Application Programming Interface*, interface de programação de aplicações) é o conjunto de regras que um software publica para que outro software possa usá-lo. A API diz *o que* pode ser pedido, *como* pedir e *o que* volta como resposta. Ela não diz como o pedido é atendido por dentro.

Essa separação é a ideia central da disciplina. De um lado fica a **interface**, que é um contrato público e estável. Do outro fica a **implementação**, que é privada e pode mudar. Enquanto o contrato é respeitado, quem usa a API não percebe a troca de banco de dados, de linguagem ou de servidor.

> **ANALOGIA**
> Num restaurante, o cardápio é a interface: lista o que pode ser pedido e o formato do pedido. A cozinha é a implementação. O garçom leva o pedido e traz o prato; o cliente nunca entra na cozinha. Se o restaurante troca o fogão, o cardápio continua o mesmo.

Três tipos de API que você já usa

Tipo	Como é chamada	Exemplo
API de biblioteca	Chamada de método dentro do mesmo programa	`List.of(1, 2, 3)` na biblioteca padrão do Java
API do sistema operacional	Chamada de sistema feita pelo programa ao SO	Abrir um arquivo, criar uma thread
API web	Mensagem pela rede, em geral HTTP	`GET /api/v1/services` no ServiceHub

Nesta disciplina tratamos de **APIs web**: um programa cliente envia uma requisição HTTP pela rede, e o servidor devolve uma resposta, normalmente em JSON. O cliente pode ser um site, um aplicativo de celular, outro sistema ou um teste automatizado.

3. Por que as APIs importam

↑ conteúdo

Integração

Sistemas escritos em linguagens e épocas diferentes conversam por um formato comum, sem que um precise conhecer o código do outro.

Reuso

A mesma regra de negócio, implementada uma vez, atende ao site, ao aplicativo e aos parceiros.

Encapsulamento e segurança

A API expõe só o necessário. Tabelas, senhas e regras internas ficam atrás dela, o que reduz a superfície de ataque.

Evolução independente

Times diferentes mudam o front-end e o back-end em ritmos diferentes, desde que o contrato seja mantido.

Há também o lado de negócio: muitas empresas vendem capacidades técnicas como produto, por meio de APIs públicas de pagamento, mapas ou mensagens. Para quem desenvolve, isso significa que o contrato da API é tão importante quanto o código que a implementa.

4. Como chegamos aqui: de RPC a REST

↑ conteúdo

Cada geração de tecnologia de integração resolveu um limite da anterior. Conhecer essa história ajuda a entender por que o REST é o padrão das APIs web e por que ele não é a única opção.

RPC: chamar uma função em outra máquina

RPC (*Remote Procedure Call*, chamada de procedimento remoto) faz uma chamada pela rede parecer uma chamada de função local. As primeiras soluções eram proprietárias: cada fabricante tinha seu protocolo, e cliente e servidor ficavam fortemente acoplados.

XML-RPC e SOAP: padronizar a mensagem

O **XML-RPC**, do fim dos anos 1990, padronizou a chamada remota usando XML como formato e HTTP como transporte. O **SOAP** evoluiu essa ideia com tipos complexos, um envelope obrigatório e uma descrição formal do serviço em **WSDL** (*Web Services Description Language*). Ele ainda é comum em bancos, governo e sistemas legados, mas é verboso. Um pedido simples ocupa várias linhas:

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Header/>
  <soap:Body>
    <GetUser>
      <userId>42</userId>
    </GetUser>
  </soap:Body>
</soap:Envelope>
```

Em REST, o mesmo pedido é `GET /users/42`, e a resposta é um JSON curto.

REST: usar a própria Web

Em 2000, Roy Fielding descreveu em sua tese de doutorado o estilo arquitetural **REST** (*Representational State Transfer*). REST não é um protocolo nem um formato: é um conjunto de restrições que aproveita o que o HTTP já oferece, como métodos, status codes, cache e URIs. A seção 5 detalha essas restrições.

GraphQL e gRPC: casos específicos

Em meados da década de 2010 surgiram duas alternativas que atacam limites do REST em cenários específicos. O **GraphQL**, publicado pelo Facebook, deixa o cliente escolher exatamente os campos que quer numa única consulta, o que evita trazer dados demais ou fazer várias chamadas. O **gRPC**, publicado pelo Google, volta à ideia de RPC com mensagens binárias e HTTP/2, e é muito usado entre microsserviços que precisam de alto desempenho.

Critério	SOAP	REST	GraphQL	gRPC
Natureza	Protocolo com especificação formal	Estilo arquitetural sobre HTTP	Linguagem de consulta	Framework de RPC
Formato	XML com envelope	JSON, em geral	JSON	Binário (Protocol Buffers)
Contrato	WSDL	OpenAPI (opcional, recomendado)	Schema GraphQL	Arquivo `.proto`
Uso típico	Integrações corporativas e legadas	APIs web públicas e privadas	Front-ends com telas variadas	Comunicação entre serviços

PARA LEMBRAR

Nenhuma tecnologia substituiu as outras. A escolha depende de quem consome a API, do volume de dados e das exigências de contrato. Neste semestre usamos REST porque ele é o padrão das APIs web e porque aproveita o HTTP que você vai estudar na Semana 02.

5. O estilo REST

↑ conteúdo

Em REST, tudo gira em torno de **recursos**: coisas do domínio que têm identidade, como um usuário, um serviço ou uma ordem. Cada recurso tem um endereço, a **URI**. O cliente nunca recebe o recurso em si, e sim uma **representação** dele, por exemplo um documento JSON com seus dados naquele momento. Daí o nome: transferência de estado representacional.

As seis restrições de Fielding

Restrição	O que exige	O que ganhamos
Cliente-servidor	Interface e armazenamento de dados ficam separados em cliente e servidor.	Cada lado evolui sozinho.
Sem estado (*stateless*)	Cada requisição carrega tudo o que o servidor precisa para atendê-la. O servidor não guarda sessão do cliente entre requisições.	Escalar é fácil: qualquer instância atende qualquer requisição.
Cache	Respostas indicam se podem ser guardadas e por quanto tempo.	Menos tráfego e respostas mais rápidas.
Interface uniforme	Recursos identificados por URI, manipulados por representações, com mensagens autodescritivas (métodos e status padronizados).	Qualquer cliente HTTP sabe conversar com a API.
Sistema em camadas	O cliente não sabe se fala com o servidor final ou com um intermediário (proxy, balanceador, gateway).	Segurança e escala sem mudar o cliente.
Código sob demanda (opcional)	O servidor pode enviar código executável ao cliente.	Raro em APIs; citado por completude.

JSON, o formato das representações

O **JSON** (*JavaScript Object Notation*) é um formato de texto com objetos (`{ }`), listas (`[ ]`), textos, números, booleanos e `null`. É leve, legível e suportado por todas as linguagens. Uma representação de serviço no ServiceHub se parece com isto:

```
{
  "id": 12,
  "title": "Instalação elétrica residencial",
  "category": "ELETRICA",
  "price": 180.00,
  "active": true
}
```

CONFUSÃO COMUM

"Usa JSON e HTTP" não torna uma API REST. Uma API com rotas como `POST /getUser` ou `GET /deleteService?id=3` usa HTTP como simples transporte, mas quebra a interface uniforme: o verbo está na URI e o método HTTP perdeu o significado.

6. Quem pode usar a API: visibilidade

↑ conteúdo

Uma das primeiras decisões de arquitetura é definir quem pode acessar a API. Essa decisão muda autenticação, documentação, versionamento e o cuidado com mudanças.

Visibilidade	Quem acessa	Exigências típicas
Privada	Só sistemas da própria organização, por exemplo serviços internos que conversam entre si.	Mudanças combinadas entre times; documentação interna.
De parceiros	Empresas com acordo formal.	Autenticação forte, contrato de nível de serviço, aviso prévio de mudanças.
Pública	Qualquer desenvolvedor que se cadastre.	Documentação exemplar, limites de uso, estabilidade do contrato e versionamento cuidadoso.

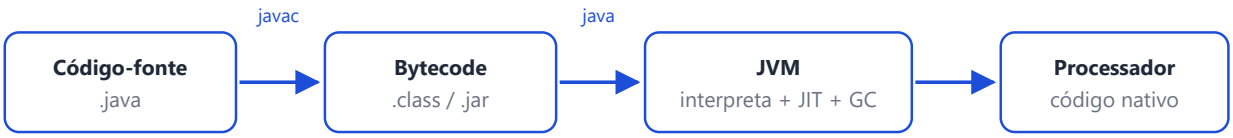
A visibilidade não é fixa: muitas APIs nascem privadas e se tornam públicas. Por isso vale projetar desde o início como se outras pessoas fossem ler o contrato. O ServiceHub tem rotas públicas, como a listagem de serviços, e rotas protegidas por login.

7. O ecossistema Java

↑ conteúdo

A JVM

O compilador `javac` não gera instruções para um processador específico. Ele gera **bytecode**, um formato intermediário que roda na **JVM** (*Java Virtual Machine*). A JVM carrega as classes, interpreta o bytecode e, para os trechos mais executados, compila para código nativo durante a execução, com o compilador **JIT** (*just-in-time*). Um **coletor de lixo** (*garbage collector*) libera a memória de objetos que não são mais usados.



Do código-fonte à execução: compila-se uma vez para bytecode, e a JVM de cada plataforma executa.

Para APIs, a JVM importa por três motivos. A aplicação fica mais rápida depois de alguns minutos de uso, quando o JIT já otimizou os trechos quentes. O coletor de lixo pode causar pequenas pausas. E a memória disponível para a JVM precisa combinar com o limite do container, tema da Semana 11.

Java 21

Usamos o **Java 21**, uma versão LTS (*long-term support*, suporte de longo prazo). Dele vêm recursos que aparecem no projeto, como os `record` (classes imutáveis de dados, ótimas para DTOs) e o `switch` com padrões.

Maven

O **Maven** gerencia o build. O arquivo `pom.xml` declara o projeto e suas dependências por **coordenadas**: `groupId`, `artifactId` e `version`. O Maven baixa as bibliotecas, compila, roda os testes e empacota o `.jar`, seguindo um ciclo de vida de fases (`compile`, `test`, `package`, `verify`, `install`). O **Maven Wrapper** (`mvnw` e `mvnw.cmd`) garante que toda a equipe use a mesma versão do Maven sem instalá-lo.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
```

8. Spring e Spring Boot

↑ conteúdo

Inversão de controle e injeção de dependência

O **Spring Framework** é, antes de tudo, um **contêiner de inversão de controle**. Em vez de cada classe criar os objetos de que precisa, o Spring cria esses objetos, chamados **beans**, e os entrega a quem pede. Esse mecanismo se chama **injeção de dependência**.

Sem injeção

```
class PedidoService {
  private final Repositorio repo =
      new RepositorioPostgres();
}
```

A classe escolhe sozinha a implementação. Trocar o banco ou testar com um dublê exige mudar o código.

Com injeção

```
@Service
class PedidoService {
  private final Repositorio repo;
  PedidoService(Repositorio repo) {
    this.repo = repo;
  }
}
```

A classe declara do que precisa no construtor. O Spring entrega a implementação certa, e o teste pode entregar outra.

Essa ideia aparece em todo o semestre: controladores recebem serviços, serviços recebem repositórios, e os testes da Semana 10 trocam dependências reais por dublês.

O que o Spring Boot acrescenta

Configurar o Spring à mão exige muitas declarações. O **Spring Boot** segue o princípio de *convenção sobre configuração*:

- **Auto-configuração:** o Boot olha as bibliotecas presentes no projeto e configura os beans comuns sozinho, como o servidor web, a conexão com o banco e a conversão para JSON.
- **Starters:** dependências agrupadas por recurso. No Spring Boot 4, cada recurso tem o seu starter, como `spring-boot-starter-webmvc`, `-data-jpa`, `-flyway`, `-validation` e `-security`.
- **Servidor embutido:** o Tomcat vem dentro do `.jar`. A API sobe com `java -jar app.jar`, sem instalar servidor de aplicação.

PEGADINHA DO SPRING BOOT 4

Adicionar só a biblioteca, por exemplo `flyway-core`, não ativa a auto-configuração. É preciso o starter do recurso, `spring-boot-starter-flyway`. Quando um recurso "não faz nada" e não aparece erro, confira primeiro se o starter está no `pom.xml`.

Os módulos que vamos usar

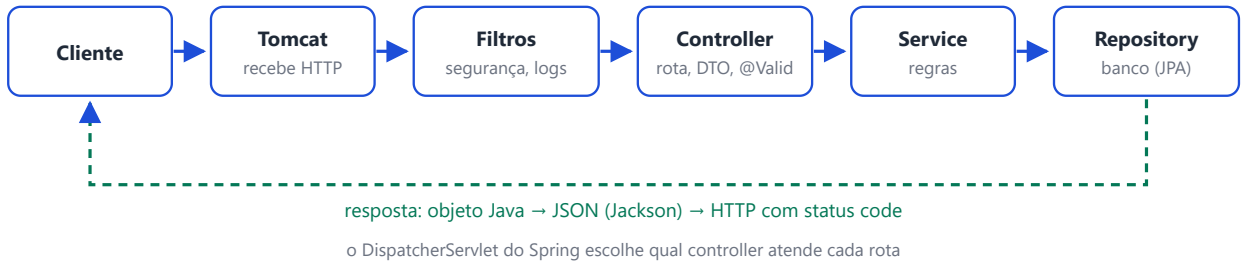
Módulo	Para que serve	Semana
Spring Web MVC	Controladores REST, conversão de JSON, tratamento de requisições	3–5
Spring Data JPA	Repositórios e acesso ao banco sem SQL repetitivo	4
Flyway	Migrações versionadas do schema	4

Módulo	Para que serve	Semana
Bean Validation	Regras de validação declaradas nos dados de entrada	4 e 8
Spring Security	Autenticação, autorização e JWT	8–9
Actuator	Saúde e métricas da aplicação	4 e 15

9. O caminho de uma requisição

↑ conteúdo

Entender por onde uma requisição passa ajuda a saber onde colocar cada tipo de código e onde procurar um erro.



Ida e volta de uma requisição numa API Spring Boot em camadas.

Camada	Responsabilidade	Não deve
Controller	Receber a requisição, validar o formato da entrada, chamar o serviço e montar a resposta HTTP.	Conter regra de negócio ou acessar o banco direto.
Service	Aplicar as regras de negócio e controlar a transação.	Conhecer detalhes de HTTP, como status codes.
Repository	Ler e gravar no banco.	Decidir regra de negócio.

Essa divisão vem de um princípio antigo de arquitetura: separar responsabilidades para que cada parte mude por um único motivo. Ela será retomada na Semana 03.

10. Nomeando recursos

↑ conteúdo

A URI identifica o recurso; o método HTTP diz a ação. Por isso URIs usam **substantivos**, e não verbos.

Bem formadas

```
GET    /api/v1/services
GET    /api/v1/services/12
POST   /api/v1/services
PATCH  /api/v1/service-requests/31/status
GET    /api/v1/services/12/reviews
```

Antipadrões

```
GET  /getService?id=12
POST /createNewService
GET  /service/delete?id=12
POST /services/doSearch
PUT  /UpdateServiceData
```

- Coleções no plural, em minúsculas, com hífen entre palavras: `/service-requests`.
- Identificador depois da coleção: `/services/12`.
- Hierarquia só quando há relação real: as avaliações de um serviço ficam em `/services/12/reviews`.
- Filtros, ordenação e paginação vão na query string: `/services?category=ELETRICA&page=0`.
- Versão no caminho, desde o primeiro endpoint: `/api/v1`.

11. O projeto do semestre

↑ conteúdo

O **ServiceHub** é uma plataforma que liga clientes a prestadores de serviço. O prestador publica serviços, o cliente contrata, a ordem de serviço percorre etapas até ser concluída, e o cliente avalia. O domínio é o mesmo do primeiro ao último dia, e cada semana acrescenta uma camada à API.

Entidade	O que representa
Usuário	Quem usa a plataforma, com um papel: CLIENT, PROVIDER ou ADMIN.
Serviço	Uma oferta publicada por um prestador, com categoria e preço.
Ordem de serviço	A contratação de um serviço por um cliente, com estado: PENDING, ACCEPTED, IN_PROGRESS, COMPLETED ou CANCELLED.
Avaliação	Nota e comentário do cliente depois que a ordem é concluída.

O contrato completo, com schema do banco, endpoints e regras de negócio, está no **Mapa de Construção**, entregue pelo professor. Ele é a especificação que a API precisa cumprir.

O semestre em blocos

Semanas	Bloco	O que a API ganha
1–3	Fundamentos e setup	Domínio, endpoints, projeto Spring Boot, Git
4–6	Persistência e contrato	JPA, Flyway, erros padronizados, OpenAPI
7	1ª VA	Prova e checkpoint do projeto
8–10	Segurança e testes	Validação, JWT, papéis, testes com banco real

Semanas	Bloco	O que a API ganha
11–12	Containers	Dockerfile e Docker Compose
13	2ª VA	Prova
14–18	Produção e governança	Entrega, métricas, logs, banca com a bancada de testes
19	3ª VA	Prova

Cada bloco termina com uma tag no repositório da equipe (`etapa-1`, `etapa-2`…), para que a evolução da API possa ser mostrada na banca.

12. Construindo com IA

↑ conteúdo

Um assistente de IA gera código rápido, mas não sabe o que o seu sistema precisa fazer, não conhece o seu contrato e erra com a mesma confiança com que acerta. O trabalho de engenharia continua seu. Neste semestre seguimos sempre quatro movimentos:

Movimento	O que você faz
Especificar	Diz o objetivo, o contexto, as restrições e como saber que ficou pronto.
Gerar	Pede à IA um plano primeiro e o código depois, em passos pequenos.
Revisar	Lê o que veio, confere contra o contrato e rejeita o que não entender.
Provar	Roda um teste ou uma chamada que mostra que funciona. Sem prova, não está pronto.

Prompt fraco e prompt bom

Fraco

```
me explica o que é REST
```

A resposta vem genérica, pode misturar conceitos e não se conecta ao que você está construindo.

Bom

```
Estou estudando APIs REST para um projeto
em Java 21 e Spring Boot 4. Explique as
seis restrições de Fielding, uma por vez,
com um exemplo de API de agendamento de
serviços. Depois me faça 3 perguntas para
eu verificar se entendi. Não use exemplos
com SOAP.
```

Dá contexto, delimita o assunto, pede exemplo concreto e transforma a IA em tutora, não em resposta pronta.

O prompt desta semana

A entrega da semana é o repositório com um README que explica o problema do ServiceHub com as palavras da equipe. Use a IA como revisora, não como autora:

```
Contexto: somos uma equipe de Engenharia de Software construindo
a API ServiceHub (Java 21, Spring Boot 4, PostgreSQL). O domínio:
clientes contratam serviços de prestadores; a ordem passa por
PENDING, ACCEPTED, IN_PROGRESS e COMPLETED; o cliente avalia.

Tarefa: revise o rascunho de README abaixo. NÃO reescreva o texto.
Aponte, em lista:
1. trechos ambíguos sobre quem faz o quê (cliente, prestador, admin);
2. termos técnicos usados de forma errada;
3. o que falta para alguém de fora entender o problema.

Rascunho:
<cole aqui o texto da equipe>
```

O que a IA costuma errar nesta etapa

- Dizer que REST é um protocolo, ou que qualquer API com JSON é REST.
- Inventar datas, autores ou números para "enriquecer" a explicação.
- Sugerir versões antigas: Spring Boot 2 ou 3, Java 8 ou 17, pacotes `javax.*` em vez de `jakarta.*`.
- Reescrever o seu texto inteiro quando você pediu revisão, apagando a voz da equipe.

Checklist de revisão

☐ Cada afirmação técnica da resposta bate com esta apostila ou com a documentação oficial?

☐ As versões citadas são Java 21 e Spring Boot 4?

☐ O README continua escrito com as palavras da equipe?

☐ Os papéis CLIENT, PROVIDER e ADMIN estão descritos sem contradição?

Prova

Peça a um colega de outra equipe que leia o README sem ajuda e explique de volta o problema do ServiceHub. Se ele explicar certo, o README cumpre o papel.

13. Glossário

↑ conteúdo

API	Conjunto de regras que um software publica para ser usado por outro.
Contrato	A parte pública de uma API: o que pode ser pedido, em que formato, e o que volta.
Recurso	Algo do domínio com identidade própria, endereçado por uma URI.
Representação	Os dados de um recurso num formato, como JSON, num dado momento.
URI	Identificador de um recurso, como `/api/v1/services/12`.
Stateless	Sem estado: o servidor não guarda sessão do cliente entre requisições.
SOAP	Protocolo de troca de mensagens XML com envelope e contrato WSDL.
JSON	Formato de texto para dados estruturados, padrão das APIs REST.
JVM	Máquina virtual que executa o bytecode Java.
Bytecode	Formato intermediário gerado pelo compilador Java.
JIT	Compilador que transforma bytecode em código nativo durante a execução.
LTS	Versão com suporte de longo prazo.
Maven	Ferramenta de build e de gerenciamento de dependências.
Bean	Objeto criado e gerenciado pelo contêiner do Spring.
Injeção de dependência	Técnica em que uma classe recebe pronto o objeto de que depende.
Starter	Dependência do Spring Boot que reúne bibliotecas e configuração de um recurso.

14. Exercícios

↑ conteúdo

1. Com suas palavras, explique a diferença entre a interface e a implementação de uma API. Dê um exemplo de mudança na implementação que não afeta quem usa a API.
2. Classifique cada caso como API de biblioteca, do sistema operacional ou web: (a) um app de celular consulta o saldo bancário; (b) um programa Java ordena uma lista com `Collections.sort`; (c) um programa grava um arquivo no disco.
3. Uma empresa precisa integrar seu sistema a um órgão público que só oferece um serviço descrito em WSDL. Que tecnologia de integração está envolvida? Que vantagem e que desvantagem ela tem em relação a REST?
4. Explique a restrição "sem estado" e por que ela facilita colocar várias instâncias da API atrás de um balanceador.
5. A API abaixo é REST? Justifique e reescreva as rotas: `POST /listarServicos`, `GET /servico/apagar?id=4`, `POST /servico/novo`.

6. Descreva o caminho de `GET /api/v1/services/12` pelas camadas de uma API Spring Boot, do cliente ao banco e de volta.
7. Um colega adicionou `flyway-core` ao `pom.xml` de um projeto Spring Boot 4, mas as migrações não rodam e não aparece erro. Qual é a causa provável e como corrigir?
8. Reescreva o código abaixo usando injeção de dependência por construtor: `class AvaliacaoService { private Repo repo = new RepoPostgres(); }`.
9. Escreva um prompt bom, seguindo o movimento Especificar, para pedir à IA uma explicação de GraphQL comparado a REST. Indique o que você conferiria na resposta.
10. Quando faz mais sentido usar gRPC do que REST? Dê um cenário.

15. Para estudar

↑ conteúdo

Leituras disponíveis na Minha Biblioteca, pelo acesso institucional:

- RODRIGUES, T. N. et al. *Integração de aplicações*. Grupo A, 2020. Capítulos "Tecnologias de integração entre sistemas e aplicações" (API, REST e RESTful, JSON) e "SOAP".
- SCHILDT, H. *Java para iniciantes*. 6. ed. Bookman. Capítulo 1, "Fundamentos da linguagem Java".
- HORSTMANN, C. *Conceitos de computação com Java*. Bookman. Capítulo 1, "Introdução".
- RICHARDS, M.; FORD, N. *Fundamentos da arquitetura de software*. Alta Books, 2024. Capítulo 1, "Introdução".

Referências abertas:

- FIELDING, R. T. *Architectural Styles and the Design of Network-based Software Architectures*. Tese (Doutorado), University of California, Irvine, 2000. Capítulo 5.
- Documentação oficial do Spring Boot: `docs.spring.io/spring-boot`.